



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY A INFORMAČNÍCH TECHNOLOGIÍ

FACULTY OF ELECTRICAL ENGINEERING AND INFORMATION TECHNOLOGY

GENERÁTOR VELKÝCH PRVOČÍSEL

LARGE PRIME NUMBER GENERATOR

STUDIE PROBLEMATIKY

STUDIES OF THE PROBLEM

AUTOŘI PRÁCE Daniel Prachař, Matěj Oszelda, Dalibor Rada
AUTHORS

LEKTOR Ing. Petr Blažek
LECTOR

BRNO 2022

Obsah

Úvod a popis projektu	3
Cíle projektu, kterých má být dosaženo	4
Teorie	4
Aktuální fáze projektu, budoucí řešení	6
Přílohy	9
Závěr.....	9
Literatura	9

Úvod a popis projektu

Projekt se věnuje problematice generování velkých prvočísel a následné testování těchto, i jiných, velkých prvočísel pomocí matematických operací, a to za použití různých, minimálně dvou, algoritmů.

Zahrnuje teoretický úvod do oblasti prvočísel, jejich hlavní vlastnosti, problém spojený se zpětnou faktORIZací a potřebu v moderních kryptografických systémech. Podkapitola generování prvočísel již konkrétně míří na algoritmy, které se u generování mohou využívat, a které byly nakonec využity. S tím je spjatá podkapitola s testováním prvočíslnosti, konkrétně druhy testů prvočíslnosti a jejich popis.

Projekt dále zakomponovává možnost výpisu všech prvočísel po zadané číslo uživatelem za účelem zjištění prvočísel. Využívá práci se soubory, přesněji pak využití dat jako vstup ze souboru pro test prvočíslnosti, popř. export prvočísla do jiného samostatného souboru pro možné další využití. Nakonec je aplikace schopná zaznamenávat probíhající události do logu, a to včetně výpisu doby trvání generování prvočísel a i jejich testování.

Poslední nejobsáhlejší kapitola obsahuje samotnou studii doposud rozpracovaného projektu v programovacím jazyce Python a další kroky potřebné k úspěšnému dokončení projektu.

Cíle projektu, kterých má být dosaženo

1. Vygenerování velkého prvočísla
2. Otestování, zdali se jedná o prvočíslo, pomocí testů prvočíselnosti a matematických operací
3. Přijmutí uživatelského vstupu (čísla) a následné vypsání všech prvočísel pro daný vstup
4. Exportování výsledků do souboru
5. Importování čísel ze souboru a následné otestování prvočíselnosti
6. Zaznamenání probíhajících událostí do logu
 - a. Čas probíhajících operací
 - b. Doba trvání generování
 - c. Doba trvání testování prvočíselnosti

Teorie

Prvočísla a jejich vlastnosti

Prvočíslo je číslo přirozené, větší než 1, které lze dělit jen jedničkou a samo sebou. Každé číslo, které není prvočíslo nazýváme číslem složeným. Každé složené číslo lze vyjádřit jako součin prvočísel procesem faktorizace. Prvočísel je nekonečně mnoho, ale pro kryptografii mají význam i některé jejich specifické se zvláštními vlastnostmi:

- Mersennova prvočísla (M_p) a patří sem čísla, pro která platí: $M_p = 2^p - 1$, kde p je prvočíslo
- Sophie-Germain prvočísla (SG_p), pro která platí vztah: $p = 2SG_p + 1$, kde p je prvočíslo
- Bezpečná prvočísla (B_p), pro která platí vztah: $B_p = 2p + 1$, kde p je prvočíslo

Důležité vlastnosti prvočísel

Pro každé přirozené číslo $n > 1$ platí, že v intervalu $(n, 2n)$ existuje alespoň jedno prvočíslo

Jestliže p je prvočíslo a $0 < a < p$ je celé číslo, pak $a^p - a$ je dělitelné p (důkaz Malá Fermatova věta)

Faktorizace

Pojem faktorizace označuje problém rozložení čísla na součin menších čísel. Tzn. pokud jakékoliv kladné celé číslo vydělíme dělitelem (b), kdy $b \geq 2$. Pokud je zbytek po dělení roven 0, pokračujeme dělením. Pokud zbytek není roven 0, dělitele zvětšíme vždy o 1 a mezivýsledek vydělíme zvětšeným dělitelem. Dělíme tak dlouho, dokud se číslo nerovná 1. Rozklad celého čísla na prvočinitele je považován za velmi náročný, při velkých číslech se proto využívá tato neovladatelnost, kdy není znám žádný efektivní algoritmus, a je proto využíván u některých kryptografických metod, např. algoritmus RSA.

Generování prvočísel

Nejjednodušší způsob generování velkých prvočísel je vygenerovat velké kladné liché číslo a následně otestovat, zda je prvočíslo. Tento způsob není moc praktický pro algoritmy, kde je potřeba pracovat s velmi velkými prvočísly.

Testování prvočíselnosti

Test prvočíselnosti je polynomiálním problémem. Jeden z nejjednodušších způsobů je algoritmus Trial division, zde zkoušíme testované číslo dělit čísly větší než 1 až po testované číslo n . Pomocí tohoto testu zjistíme jistě, zda je číslo opravdu prvočíslem. Pokud bude výsledek alespoň jednoho dělení beze zbytku, číslo není prvočíslo. V tomto případě je počet operací zbytečně velký, pokud rozsah dělitelů zmenšíme o všechna sudá čísla a maximální dělitel v rozsahu bude $\sqrt{n}$, eliminujeme mnoho zbytečných kroků.

Další možností tohoto testu je algoritmus Wheel factorization. Zde provádíme stejné akce, jen opět eliminujeme velký počet dělitelů. Číslo dělíme jen několika prvními prvočísly a násobky těchto prvočísel z rozsahu dělitelů odebereme, poté číslo dělíme jen čísly nesoudělnými s těmito prvočísly až po $\sqrt{n}$.

- **Pravděpodobnostní testy**

Tyto testy je lehčí spočítat pro vysoké čísla, ale není úplně jisté, že je číslo opravdu prvočíslo.

1. Fermatův test

Tento test neumí rozlišit prvočísla od Carmichaelových čísel, což jsou speciální složená čísla. Základem tohoto testu je malá Fermatova věta. $1 \leq y \leq n - 1$, kde n je prvočíslo a y je číslo nesoudělné s n . Potom provedeme jednoduchou operaci $a^{n-1} \equiv 1 \pmod{n}$, pokud toto platí, není úplně jisté, že je číslo opravdu prvočíslem. Proto můžeme operaci opakovat s jiným a , abychom zvětšili šanci, že je číslo opravdu prvočíslem.

2. Miller-Rabinův test

Tento test je hodně používaný. Zde si n vyjádříme jako $n = 2^s r + 1$ kde r je liché. Zvolíme náhodné a $1 \leq a \leq n - 1$, dále testujeme zda $a^r \equiv 1 \pmod{n}$, pokud toto platí, pokračujeme, pokud ne, zkusíme $a^{2^i r} \equiv n - 1 \pmod{n}$ pro $0 \leq i \leq s - 1$, pokud platí, jdeme dál. Pokud tento bod neplatí, číslo je složené. Test můžeme zkoušet pro jiné a . Chybnost tohoto testu je 25 %.

- **Testy dokazující prvočíselnost**

Testy dokazující prvočíselnost jsou schopny dokázat prvočíselnost kladného celého čísla. Avšak testy tohoto typu jsou výpočetně náročnější, proto je vhodné kandidáta na prvočíslo nejprve otestovat pomocí testu pravděpodobnostního. Hlavním zástupcem v této skupině je Lucas-Lehmerův test, který využívá Mersenovo číslo ve tvaru $M_p = 2^p - 1$, kdy p je prvočíslo, pokud:

$$u_0 = 4 \text{ a } u_{k+1} = (u_k^2 - 2) \bmod n, \text{ tak } u_{s-2} = 0.$$

Aktuální fáze projektu, budoucí řešení

Při spuštění projektu se nejprve zobrazí úvodní text, který přivítá uživatele. Program uživateli nabídne menu akcí. Menu je realizováno pomocí cyklu while s kombinací if/else bloku. Akci uživatel vybere zadáním čísla 1-5 na klávesnici. Jiný zadaný vstup zobrazí chybový text. Uživatel má na výběr z 5 možných akcí:

1. Vygenerování velkého prvočísla, otestování prvočísla a export do souboru
2. Otestování zadaného prvočísla
3. Zadání čísla a následné vypsání všech prvočísel, které obsahuje
4. FaktORIZACE složeného čísla
5. Informace o projektu

1. Vygenerování velkého prvočísla, otestování prvočísla a export do souboru

Při vybrání této možnosti v menu se prvně vygeneruje velké náhodné číslo. Poté se otestuje, jestli je číslo prvočíslo, pokud je číslo složené, program se ukončí. Tato část kódu bude nakonec nejspíš vypadat úplně jinak. Kód nakonec upravíme tak, že vygenerujeme prvočíslo a budeme moci vybrat jeden ze dvou algoritmů na testování, pokud je číslo opravdu prvočíslem. Poté bude možnost číslo exportovat do souboru.

2. Otestování zadaného prvočísla

Při vybrání akce 2 se uživateli zobrazí výběrové podmenu. Podmenu je rovněž realizováno pomocí cyklu while a if/else bloku. Dává uživateli možnost vybrat jakým způsobem chce vložit dané číslo pro otestování a jestli chce výsledek exportovat do textového souboru. Uživatel vybere akci zadáním čísla 1-3 z klávesnice. Jiný zadaný vstup zobrazí chybový text. Struktura podmenu:

1. Chceš zadat číslo z konzole
2. Chceš zadat číslo ze souboru
3. Chceš exportovat výsledek do souboru

Při výběru akce 1:

Program požádá uživatele o vložení čísla do konzole programu. Program číslo zadané uživatelem načte a nejdříve zkontroluje jestli je číslo větší než 2, aby mohlo být provedeno otestování prvočísla. Jestliže je vstup v pořádku, program otestuje zadané číslo Brute Forcem. Operace je vytvořena pomocí cyklu for, který zkoumá jestli se modulo zadaného čísla = 0. Pokud ano, program vypíše, že se nejedná o prvočíslu, ale o číslo složené. Pokud ne, program zmenší prvočíslu o 1 a opět zkoumá, jestli se modulo daného čísla = 0. Jestliže po projití všech možností program nezjistil, že se jedná o složené číslo, tak se dané číslo dá označit za prvočíslu. Výsledek vypíše na obrazovku.

Při výpisu, že se jedná o složené číslo, program je zde zcela ukončen metodou exit(), znemožňuje tak uživateli pokračovat v programu i nadále. Jestliže by zde metoda exit() nebyla, výpis se chová divně, tuto funkcionalitu budeme nuceni opravit, obzvláště pokud bychom chtěli kód nějakým způsobem používat.

Hlavním problémem zde je, že nevyužíváme efektivní algoritmy pro otestování, nýbrž Brute Force. Problematiku budeme později řešit algoritmy založené na matematických testech prvočíslnosti (pravděpodobnostní i skutečné). Dle přiložené literatury bychom chtěli využívat Miller-Rabinův test a Lucas-Lehmerův test.

Při výběru akce 2:

Program číslo načte přímo z textového souboru, do kterého jej uživatel může zadat. Soubor je uložen ve složce s programem a nazývá se random_cislo.txt. V souboru je uloženo počáteční číslo pro testování našeho programu. Po načtení čísla ze souboru se soubor automaticky zavře. Dále program pokračuje jako při výběru akce 1. Otestuje prvočíslu a zobrazí na obrazovku, jestli se jedná o číslo složené nebo o prvočíslu.

Při výběru akce 3:

Po výběru volby 3 program umožní výsledky exportovat do souboru, a to jak čísla zadaného konzolí, tak i čísla zadaného souborem. Soubor se vytvoří ve složce programu s názvem výsledky_test.text. Při úspěšném exportu program vytiskne na obrazovku hlášku: Všechny hodnoty byly úspěšně zapsány.

Program se vždy zeptá jestli chce uživatel pokračovat v tomto podmenu. Jestliže uživatel zadá volbu ano, program mu umožní pokračovat. Při zadání jakéhokoliv jiného slova se program přepne do hlavního menu, kde se uživatele znova zeptá jestli chce pokračovat. Při zadání volby ano program setrvá v hlavním menu. Při zadání jiného textu se program kompletně ukončí s děkovným textem.

3. Zadání čísla a vypisání všech prvočísel po zadané číslo

Program požádá uživatele o zadání čísla do konzole. Pomocí cyklu for, cyklu while a bloku if/else vypisuje všechna prvočísla až po číslo zadané uživatelem. Jestliže uživatel zadá prvočíslu, program vypíše seznam prvočísel včetně čísla zadaného uživatelem. Výpis prvočísel funguje na zkoumání jestli se modulo = 0. Následně program provede celočíselné dělení. Jestliže je počet kroků menší než 1, zadané číslo je prvočíslu a vypíše se. Jestli ne, číslo prvočíslu není a počet kroků se zvětší o 1 a zároveň zvětšíme i dělitele o 1. Program po skončení vypíše posloupnost prvočísel do konzole, zároveň se však výsledky exportují do textového souboru, který je uložen ve složce s programem s názvem seznam_prvocisel.txt

Hlavním problémem je export seznamu prvočísel, který zatím nefunguje a vypíše se pouze první prvočíslu => realizace: čísla uložíme do listu nebo jiné datové struktury a následně zapíšeme do souboru.

4. FaktORIZACE čísel

Po uživateli je požadováno zadat vstupní číslo. Jestliže uživatel zadá prvočíslu, program dané prvočíslu vypíše. Jestliže uživatel zadá číslo složené program, provede faktORIZACI tj. rozklad složeného čísla na jednotlivé prvočísla, ze kterého se skládá. FaktORIZACE je realizovaná cyklem while, který vypisuje čísla a blokem if/else, který zkoumá jestli se modulo čísla = 0.

5. Informace o projektu

Při výběru volby 5 se uživateli zobrazí základní info o projektu: název, autoři, datum vytvoření a další

Při skončení jakékoliv akce se program vždy uživatele dotáže, zdali chce pokračovat. Jestliže uživatel zadá, volbu ano, program setrvá v hlavním menu a umožní pokračování. Jestli uživatel zadá jiný text, program se ukončí s děkovným textem

Přílohy

Náš kód: https://gitlab.com/dandos42/bpc-akr_python_generovani_prvocisel

Závěr

Cílem naší práce je vygenerovat velké prvočíslo a následně jej otestovat pomocí matematických operací. V teoretické části práce jsou popsány jednotlivé známé metody generování a testování prvočísel. Taktéž v programu je možnost testovat číslo, jež jsme zadali do konzole, nebo jej načetli ze souboru. Další funkcí programu je vygenerovat všechna prvočísla až po zadané číslo. V programu je zakomponována faktorizace složeného čísla na prvočísla, z kterých se zadané číslo skládá. Tento proces je více popsán v teoretické části.

Cílem naší práce je vygenerovat velké prvočíslo a následně jej otestovat. Následně prvočíslo otestovat pomocí matematických operací. V teoretické části práce jsou popsány jednotlivé známé metody generování a testování prvočísel. Taktéž v programu má být možnost testovat číslo, jež jsme zadali do konzole, nebo jej načetli ze souboru. Dalším úkolem je vygenerovat všechna prvočísla až po zadané číslo. V programu musí být zakomponována faktorizace složeného čísla na prvočísla, z kterých se zadané číslo skládá.

Literatura

- [1] OCHODKOVÁ, Eliška. Matematické základy kryptografických algoritmů. 2011.
- [2] JOYE, Marc; PAILLIER, Pascal; VAUDENAY, Serge. Efficient generation of prime numbers. In: International Workshop on Cryptographic Hardware and Embedded Systems. Springer, Berlin, Heidelberg, 2000. p. 340-354.
- [3] HAJNÝ, Jan. Přednáška předmětu BZKR – Základy kryptografie 3. [online prezentace]. VUT v Brně, 2021 [cit. 2022-10-30]. Dostupné z URL: <https://moodle.vut.cz/course/view.php?id=242381>
- [4] HAJNÝ, Jan. Přednáška předmětu BZKR – Základy kryptografie 4. [online prezentace]. VUT v Brně, 2021 [cit. 2022-10-30]. Dostupné z URL: <https://moodle.vut.cz/course/view.php?id=242381>